LAB 18

Diagnostics Stage: Automated Bias Checks before Go-Live

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Diagnostics
TSC reference	A9, K8
Duration	20 min
Folder	labs/Lab18-diagnose-bias/
Files	Lab18-diagnose-bias/diagnose_stage.py, Lab18-diagnose-bias/buggy_backtest.py, pipeline/diagnose.py, Lab18-diagnose-bias/solution/diagnose.py
Simulator page	http://localhost:8000/#/research/diagnostics?strategy=custom_lab18_biased_smoothed_momentum
Mode	Runs ONLINE with your Alpaca paper keys (.env), or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

A junior quant sends the investment committee a 'smoothed momentum' backtest on SPY with a Sharpe ratio above 4 and asks to go live. The desk responds by adding a diagnostics stage to the pipeline: every strategy - including its own ema_cross - must pass automated checks for look-ahead, cost drag, overfitting and survivorship before it may trade, even on paper.

Objective

Build `python -m pipeline diagnose` to run automated look-ahead (features and execution), cost-drag, overfitting and survivorship checks on any strategy and on the junior quant's exhibit, and write a PASS / WARN / FAIL report with a go-live verdict.

Before You Start

- Part A: start the Trading Simulator (uv run trader serve --with-worker, or docker compose up -d) and open http://localhost:8000 > Course Labs > Lab 18. Install steps: Learner Guide, 'Download, Install and Start the Trading Simulator Software'.
- Part B: open a terminal in the labs folder, activate your virtual environment and open your AI coding assistant there; the finished reference code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Part A - In the Trading Simulator

Start the simulator (uv run trader serve --with-worker, or docker compose up -d) and open http://localhost:8000. In the sidebar open Course Labs > Lab 18: the app shows the same steps with a Go button that opens each page pre-filled. Every step works offline on the bundled demo data and the built-in paper broker.

A1. Diagnose the biased exhibit

Research > Diagnostics on 'Lab 18 exhibit: biased smoothed momentum' (seeded custom strategy).

Open: http://localhost:8000/#/research/diagnostics?strategy=custom_lab18_biased_smoothed_momentum (Research > Diagnostics (Lab 18 exhibit))

Check: Look-ahead (features) FAIL: rolling(center=True) uses future bars. Verdict BLOCK.

A2. Fix it in the Strategy Builder

Open the exhibit in the Strategy Builder, change center=True to center=False, Validate, Save.

Open: http://localhost:8000/#/builder?key=custom_lab18_biased_smoothed_momentum (Strategy Builder (exhibit))

Check: The validation's look-ahead check now passes.

A3. Re-run diagnostics

Diagnose the fixed strategy; compare the same-bar vs next-bar Sharpe and the cost-drag row.

Open: http://localhost:8000/#/research/diagnostics?strategy=custom_lab18_biased_smoothed_momentum

```
Command line: smooth = data['close'].rolling(5, center=True).mean()   # BUG: uses t+1 and t+2
smooth = data['close'].rolling(5).mean()              # FIX: past prices only
```

Check: No FAIL on look-ahead; cost drag and overfitting WARN explained.

Expected in the app: A FAIL on the biased exhibit, a PASS / WARN table for the fixed version and a verdict.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

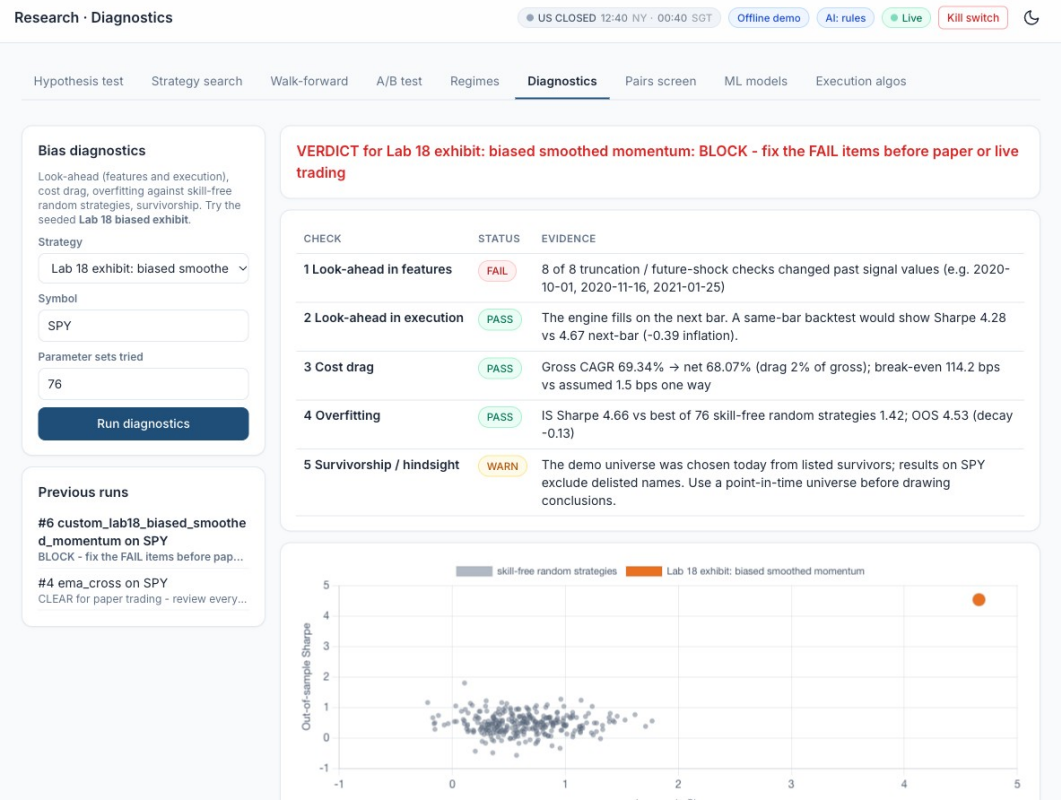


Figure: Lab 18 in the Trading Simulator (Lab18_diagnostics.png).

Part B - In Code with Your AI Assistant

The same pipeline stage in code. AI writes the code; you own the algorithm: paste each prompt into your AI coding assistant in the labs folder (the finished reference code is already in labs/Lab18-diagnose-bias/), run it, and check the result against Part A. The Practical Performance assessment uses this code path.

Step 1: Run the suspicious backtest (A9)

Run: python3 Lab18-diagnose-bias/buggy_backtest.py. Record the CAGR, Sharpe ratio and win rate.

Check: CAGR 71.52%, Sharpe 4.28, win rate 67.5% - far above anything a real SPY timing rule achieves.

Step 2: Find the problems (A9, K8)

Read the three lines marked CHECK 1 to 3 and write down what is wrong with each.

AI prompt - paste into your AI coding assistant
Review labs/Lab18-diagnose-bias/buggy_backtest.py as a model-risk reviewer. For each line marked CHECK 1 to 3, explain the bias, how it inflates the result and the correct fix. Do not run or change the file.

Check: CHECK 1 centred window (uses t+1 and t+2), CHECK 2 same-bar returns, CHECK 3 no costs.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the look-ahead checks (A9, K8)

Ask your assistant for the first part of pipeline/diagnose.py.

AI prompt - paste into your AI coding assistant
Create labs/pipeline/diagnose.py. buggy_signal(bars) reproduces the exhibit (5-day rolling mean with center=True, long when it rises); trailing_signal(bars) is the fix (trailing window). In run(strategy=None, trials=None) add check 1, look-ahead in FEATURES, using pipeline.signals.lookahead_test (25 truncation truncation and future-shock checks) on the configured strategy and on the exhibit (fn=buggy_signal); and check 2, look-ahead in EXECUTION: the strategy's Sharpe with same-bar returns (target_t x return_t) versus the pipeline's next-bar backtest, plus the exhibit fixed one problem at a time: 0 as submitted (centred, same bar), 1 trailing window same bar, 2 + next-bar execution (common.backtest.backtest), 3 + alpaca_retail costs, and SPY buy and hold, as a CAGR / Sharpe / max drawdown table. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: The exhibit fails the truncation test; the configured strategy passes.

Step 4: Build the cost, overfitting and survivorship checks (A9, K8)

Ask your assistant for checks 3-5, the report and main().

AI prompt - paste into your AI coding assistant
In labs/pipeline/diagnose.py add: check 3 cost drag - gross vs net CAGR from cost_summary(), break-even from pipeline.backtest.break_even_bps versus the assumed one-way cost (half spread + slippage of the config preset): PASS if break-even >= 3 x assumed and drag < 25% of gross; check 4 overfitting - random_strategies(spy_returns, 500, seed=1) (skill-free long/flat signals switching with 10% probability a day, traded next bar), the best in-sample Sharpe among the first N where N = the number of sets in _outputs/Lab10_grid_results.csv (default 76), compared with the strategy's 2018-2021 Sharpe, and its 2022-2025 decay; check 5 survivorship / hindsight - the universe was chosen from today's listed symbols, and an equal-weight NVDA/MSFT/AAPL 'AI basket' versus SPY. Print each check as [PASS|WARN|FAIL|INFO] check subject evidence, a VERDICT ('BLOCK - fix the FAIL items' if the strategy has a FAIL, else 'CLEAR for paper trading - review every WARN'), write _outputs/Lab18_diagnostics.csv and charts Lab18_bias_comparison.png and Lab18_overfitting.png. main(argv): --strategy, --trials, --strict (exit 1 on BLOCK). Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline diagnose --help shows --strategy, --trials and --strict.

Step 5: Run the diagnostics (A9)

Run: python -m pipeline diagnose (same as: python3 Lab18-diagnose-bias/diagnose_stage.py).

Check: ema_cross: look-ahead PASS (0 of 25), execution PASS (same-bar Sharpe 1.45 vs next-bar 0.82), cost drag PASS (gross 9.13% -> net 9.02%, break-even 135.8 bps vs 1.5 bps), overfitting WARN (IS Sharpe 0.86 vs best of 76 random 1.42; OOS 0.75), survivorship WARN. Exhibit: look-ahead FAIL (25 of 25), execution FAIL (4.28 -> 0.24). VERDICT for ema_cross: CLEAR for paper trading - review every WARN.

Step 6: Read the exhibit fixed one problem at a time (A9, K8)

Read the fix table and open _outputs/Lab18_bias_comparison.png.

Check: As submitted 71.47% / Sharpe 4.28; trailing window, same bar 66.56% / 4.68; + next-bar execution 2.23% / 0.24; + realistic costs 1.46% / 0.18; SPY buy and hold 14.14% / 0.78 - the same-bar error is the main bias.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 7: Read the overfitting and survivorship evidence (K8)

Read the INFO row and open _outputs/Lab18_overfitting.png. Then run python -m pipeline diagnose --strategy dip_buy.

Check: 500 random strategies: best IS Sharpe 1.77 -> OOS 0.55, median OOS 0.46, IS/OOS correlation -0.05, 16% have IS Sharpe > 1. AI basket CAGR 38.84% vs SPY 14.15%. dip_buy: cost drag PASS (break-even 19.5 bps), overfitting WARN (IS Sharpe 0.14, OOS 0.80).

Step 8: Use the stage as a gate and write the committee note (A9)

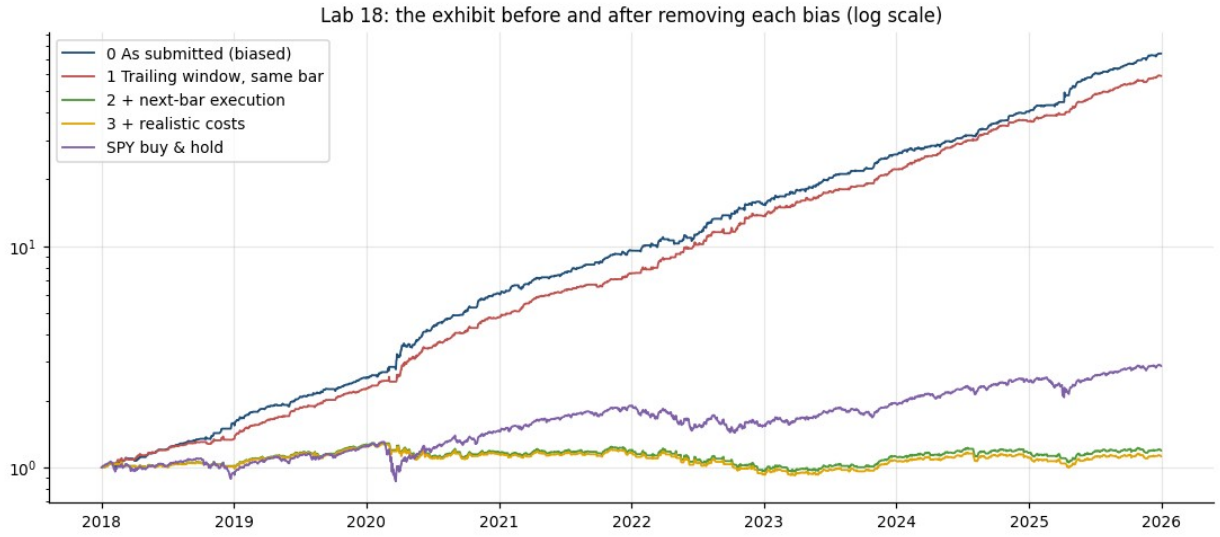
Run: python -m pipeline diagnose --strict and check the exit code (0 for ema_cross). The e2e capstone uses the same verdict to block orders.

AI prompt - paste into your AI coding assistant
Using labs/_outputs/Lab18_diagnostics.csv, write a 4-sentence note for the investment committee: why the junior quant's Sharpe of 4.28 is not credible, which automated check caught each problem, what the configured ema_cross strategy's WARN items mean, and what must happen before any live capital.

Check: Exit code 0 for ema_cross; a note that names look-ahead (features and execution), cost drag, overfitting and survivorship.

Expected Output

python -m pipeline diagnose: ema_cross PASS for look-ahead (0 of 25), execution (same-bar 1.45 vs next-bar 0.82) and cost drag (break-even 135.8 bps vs 1.5 bps), WARN for overfitting (IS 0.86 vs best of 76 random 1.42) and survivorship -> CLEAR for paper trading; exhibit FAIL: 25 of 25 look-ahead checks, Sharpe 4.28 as submitted -> 0.24 fixed -> 0.18 with costs; 500 random strategies best IS 1.77 -> OOS 0.55; AI basket 38.84% vs SPY 14.15%; files _outputs/Lab18_diagnostics.csv and charts _outputs/Lab18_bias_comparison.png and _outputs/Lab18_overfitting.png.



Lab 18 output (Lab18_bias_comparison.png), offline mode on the bundled data.

Debrief

Most 'too good to be true' results come from a short list of causes (K8): look-ahead in features or execution, overfitting, survivorship and ignored costs. Automating the checks and using the verdict as a

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

gate (A9) is what stops an unintended outcome from reaching even a paper account - and the same checks run on the desk's own strategy, not only on other people's.

Troubleshooting

Symptom	What to do
Step 1 shows a slightly different CAGR from the fix table	buggy_backtest.py drops the first NaN rows; the stage fills them with 0 - 71.52% vs 71.47%.
Random-strategy numbers differ	Change the seed or --trials and the threshold changes, but the pattern (no persistence) stays.
Is center=True really look-ahead?	Yes: a centred 5-day mean at day t uses the closes of t+1 and t+2.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.